Smart events and space scheduler

```
import datetime
from typing import List


class Event:
    def __init__(self, name, start_time, end_time, space):
        self.name = name
        self.start_time = start_time
        self.end_time = end_time
        self.space = space


    def conflicts(self, other_event):
        if self.space != other_event.space:
            return False
        return not (self.end_time <= other_event.start_time or self.start_time >=
other_event.end_time)


    def __str__(self):
        return f"{self.name} | {self.space} | {self.start_time} - {self.end_time}"
```

Smart events and space scheduler

```
class Scheduler:
    def __init__(self):
        self.events: List[Event] = []

    def add_event(self, event: Event):
        for existing in self.events:
            if event.conflicts(existing):
                print("Conflict detected with:", existing)
                return False
        self.events.append(event)
        print("Event added successfully")
        return True

    def remove_event(self, name: str):
        self.events = [e for e in self.events if e.name != name]
        print("Event removed")

    def view_events(self):
        if not self.events:
            print("No events scheduled")
            return
```

```
        for e in sorted(self.events, key=lambda x: x.start_time):
            print(e)

    def suggest_free_slots(self, space, date):
        print(f"Free slots for {space} on {date}:")
        day_start = datetime.datetime.combine(date, datetime.time(8, 0))
        day_end = datetime.datetime.combine(date, datetime.time(20, 0))

        events = [e for e in self.events if e.space == space and e.start_time.date() ==
date]
        events.sort(key=lambda x: x.start_time)

        free_start = day_start
        for e in events:
            if free_start < e.start_time:
                print(f"Free: {free_start.time()} - {e.start_time.time()}")
            free_start = max(free_start, e.end_time)

        if free_start < day_end:
            print(f"Free: {free_start.time()} - {day_end.time()}")
```

```
def parse_time(date_str, time_str):
    return datetime.datetime.strptime(date_str + " " + time_str, "%Y-%m-
%d %H:%M")



def main():
    scheduler = Scheduler()

    while True:
        print("\n===== SMART EVENT SCHEDULER =====")
        print("1. Add Event")
        print("2. View Events")
        print("3. Remove Event")
        print("4. Suggest Free Slots")
        print("5. Exit")

        choice = input("Enter choice: ")

        if choice == '1':
            name = input("Event Name: ")
```

```
    space = input("Space (Room/Hall): ")
    date = input("Date (YYYY-MM-DD): ")
    start = input("Start Time (HH:MM): ")
    end = input("End Time (HH:MM): ")

    start_time = parse_time(date, start)
    end_time = parse_time(date, end)

    event = Event(name, start_time, end_time, space)
    scheduler.add_event(event)

elif choice == '2':
    scheduler.view_events()

elif choice == '3':
    name = input("Enter event name to remove: ")
    scheduler.remove_event(name)

elif choice == '4':
    space = input("Enter space: ")
    date_str = input("Enter date (YYYY-MM-DD): ")
```

```
            date = datetime.datetime.strptime(date_str, "%Y-%m-%d").date()
            scheduler.suggest_free_slots(space, date)

        elif choice == '5':
            print("Exiting...")
            break

        else:
            print("Invalid choice")

if __name__ == "__main__":
    main()
```